[에코마케팅]마케팅 프로세스 자동화 개발자 (Python) 신입/경력사원 채용

상세요강 접수기간·방법 기업정보

■ **주요 업무**

- **광고 운영 및 마케팅 프로세스 분석**을 통한 자동화 기회 발굴 및 코드 작업
- **자동화 솔루션 개발 (Python 중심)**
- AI 및 자동화 신규 프로젝트 **아키텍처 설계 및 운영화**

■ **기술 스택 및 협업 환경**

- **개발 언어:** Python (FastAPI, Flask 등 활용 경험 우대)
- **DB:** MySQL, PostgreSQL, NoSQL(mongodb, dynamodb등)
- **AI 활용:** OpenAI API, LangChain/LangGraph 기반 LLM 연동
- **자동화:** Selenium, RPA, 크롤링 등
- **Infra:** AWS (EC2, Lambda, S3, RDS 등), Docker
- **협업 도구:** Jira, Confluence, Notion, Google Drive, Slack, MS-Teams

■ **지원 자격**

- 국내외 4년제 대학 졸업(예정)인 분
- Python을 활용한 데이터 처리 또는 자동화 경험이 있는 분
- LLM(AI) 트렌드 연구 및 관련 API 활용 경험이 있는 분

마케팅 데이터 분석가(신입)

상세요강 접수기간·방법 기업정보 추천공고

담당업무

1. 데이터 시스템 구축 및 운영

- AI 답변 데이터 추출 및 관리 시스템 구축: ChatGPT, Gemini 등 주요 생성형 AI 모델의 답변 데이터를 주기적으로 추출, 정제, 메타데이터 태깅(Tagging)하는 프로세스 및 DB(Firestore 등) 설계 및 운영 주도.
- 데이터 수집(Web Crawling), 전처리, 분석 자동화를 위한 Python 활용.
- 대규모 데이터셋 관리 및 복잡한 쿼리 작성을 위한 SQL/엑셀 활용

2.심층 분석 및 인사이트 도출

- 보고서 작성 및 전략 제안: 수집한 데이터를 기반으로 키워드를 체계적으로 분류하고, 도출된 인사이트를 바탕으로 보고서를 작성하여 고객사에 전략 제안
- 웹 분석 데이터(유입, 전환, 사용자 행동) 심층 분석 및 보고서 작성, GA4를 활용한 고객 여정 및 퍼널 분석.
- AI 모델 API 연동을 통한 데이터 추출 및 관리.
- 클라우드 환경에서 데이터 파이프라인(ETL) 구축 및 데이터베이스(RDB 또는 NoSQL, Firestore 등) 관리.
- 분석 결과를 비즈니스 관점에서 시각화하고 대시보드를 구축하여 공유

[굳센] Python 소프트웨어 개발자 채용

상세요강 접수기간·방법 기업정보

포지션 및 자격요건

포지션 (1명)

담당업무

- 반복 업무 자동화(배치/봇/스크립트) 개발 및 개선
- 웹 데이터 수집(크롤링) 파이프라인 구축 및 운영

자격요건 (필수)

- 신입/경력 무관
- Python 개발 실무 경험 해보신 분
- 크롤링 실무 경험 해보신 분

우대사항

- 자동화 개발 경험 해보신 분
- 3개월 후 연장 가능하신 분

[AI Saas] 데이터사이언티스트 / 크롤러 전문가 채용

상세요강 | 접수기간·방법 | 기업정보 | 추천공고 | IT 개발자 전문 채용관

[주요업무]

1. 데이터 수집 및 크롤링
- 국내·외 OTA(예: Booking.com, Agoda, Expedia 등) 호텔 가격 및 재고 데이터 크롤링
- 크롤링 구조 설계 (동적 페이지, 세션/쿠키, 지역·날짜·인원 조건 처리)
- 크롤링 안정성 및 정확도 개선 (차단 대응, 스케줄링, 에러 핸들링)
- 수집 데이터 정합성 검증 및 정제 파이프라인 구축

2. 다이나믹 프라이싱 모델 개발
- 호텔 객실 요금 예측 및 최적화 모델 설계
- 수요 예측, 가격 탄력성 분석, 경쟁가 반응 모델링
- Rule-based + ML/AI 혼합 구조 설계
- A/B 테스트 및 성과 측정

3. 데이터 분석 및 시스템 연동
- PMS, Channel Manager, 내부 매출 데이터 연동
- 대규모 시계열 데이터 분석
- 모델 결과를 서비스/API로 배포 및 운영

[자격요건]
- Python 기반 개발 역량
- 데이터 수집/크롤링 경험 (BeautifulSoup, Selenium, Playwright, Scrapy 등)
- SQL 및 데이터 처리 경험 (Pandas, NumPy 등)
- 머신러닝 또는 통계 모델링에 대한 기본 이해
- 실제 서비스 환경에서 코드 운영/개선 경험

함께 플랫폼을 빌드업할 Python 데이터 엔지니어 모집

상세요강 | 접수기간·방법 | 기업정보 | 추천공고 | IT 개발자 전문 채용관

Python 크롤링 데이터 엔지니어 (1명)	**담당업무** **1) 웹 크롤링 개발 및 유지보수** 웹페이지 크롤러 설계/구현 HTML 구조 변경 시 자동 보정 로직 설계 크롤링 안정성/속도 개선 작업 **2) SNS API 데이터 수집** Instagram / TikTok / YouTube 공식 API 연동 좋아요·조회수·팔로워 등 기초 데이터 수집 API Rate Limit 대응 및 예외 처리 **3) 데이터 적재 파이프라인 개발** Rolling 인플루언서 데이터 적재 일별 변화량(증감) 계산 및 저장 Python Batch/Cron 기반 자동화 구현 **4) 대량 데이터 처리** 대량 요청 처리 로직 설계 중복 데이터 필터링 및 클렌징 성능 최적화 및 안정성 확보 **5) PHP팀·서버팀 협업** PHP 개발자와 API 결과 연동 서버팀과 배치/로그 스케줄 협업 **스킬** • API, MySQL, Linux, JSON, REST API, Beautifulsoup, Requests, Selenium

[코스닥기업/킨텍스역위치]빅데이터 수집시스템 개발 및 운영자 채용합니다

상세요강 | 접수기간·방법 | 기업정보 | 추천공고 | 잡아바 추천 TOP 기업 온라인 채용관

포지션 및 자격요건

빅데이터 수집시스템 개발 및 운영 (○명)

담당업무

- 빅데이터 수집시스템 개발 및 운영
  > 빅데이터 수집시스템 개발 및 운영관리
  > 비정형 데이터 파이프라인 구축
  > 관리시스템 UI 개발
  > 시스템 모니터링 및 장애처리

자격요건

- 초대졸 이상
- IT 관련학과 졸업 및 예정자
- IT 관련업종 경력직

우대사항

- PHP, Python, Node.js, JavaScript 등 중 한 가지 이상의 언어에 숙련된 분
- Selenium, Playwright, Puppeteer, BeautifulSoup, Scrapy 등 크롤링 프레임워크 경험
- REST API, JSON, HTTP 통신 구조에 대한 이해
- 데이터 수집 로직의 예외처리 및 안정화 경험
- Linux 환경에서의 개발 및 배포 경험

Chap 1. 웹 크롤링을 위한 환경설정과 자동 검색 구현

웹 크롤링의 세계에 오신 것을 환영합니다~
제가 지금부터 자세하게 전해 드릴 테니까 차근차근 잘 배워서 꼭 능력자가 되세요~~^^
그리고 미리 당부를 드릴 내용이 있어요.
이번 챕터부터 하는 내용들은 파이썬 기본 문법들을 잘 알고 있다고 전제하고 진행을 할거예요.
그래서 혹시라도 파이썬의 기본문법이 잘 기억이 안 나시는 분들은 기본 문법 파트를 참고하면서 보면 훨씬 이해가 잘 되실 거예요. 기본 문법이 이해가 안된다고 좌절하지 말라는 의미입니다!

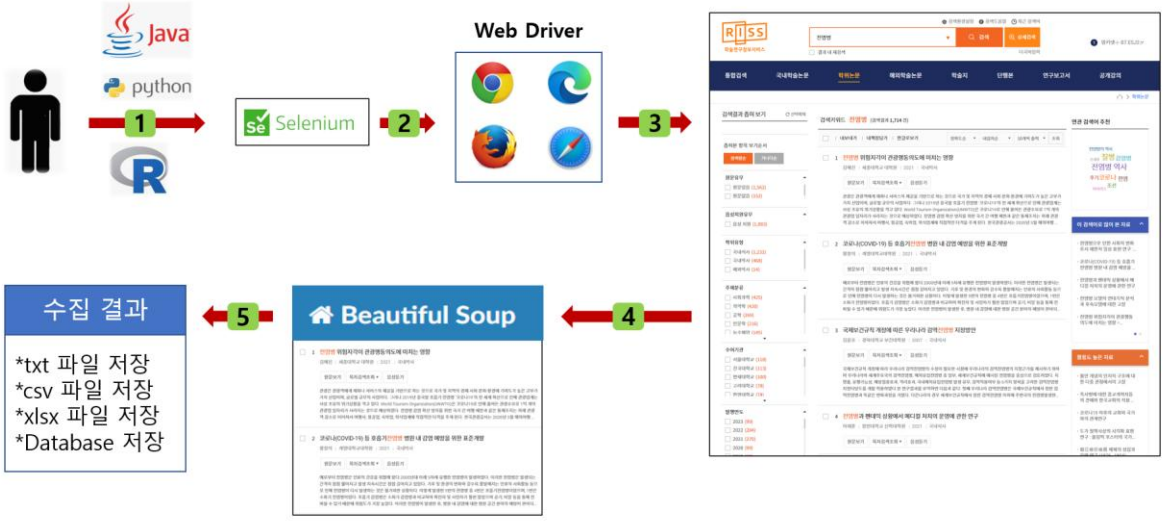
자.....그러면...우리가 이번 챕터에서 배워야 할 목표를 먼저 정리해 볼까요?

**** 이번 챕터의 학습 목표 ****
1. 웹 크롤링의 원리를 이해하고 설명할 수 있다.
2. 웹 크롤링을 위한 주요 모듈들을 설치하고 환경설정을 할 수 있다.
3. 웹사이트에서 검색 자동화를 구현할 수 있다.

위의 학습목표를 모두 공부하면 여러분들은 다양한 사이트에서 검색의 도사가 될 수 있습니다~
그럼 이제 본격적으로 시작해 볼까요?

1. 웹 크롤링의 원리

먼저 웹 크롤링을 하려면 데이터를 수집해 올 수 있도록 필요한 환경 설정이 되어야 합니다~
웹 데이터를 크롤링하는 다양한 모듈과 방법들이 있지만 이 책에서는 selenium 과 Beautiful Soup 를 사용하고 웹 드라이버로 Google 의 Chrome Driver를 사용하는 방법을 자세히 소개 드리겠습니다. 갑자기 여러가지 용어가 한꺼번에 나오니까 복잡하죠?
아래 그림을 보고 웹 크롤링을 하는 원리를 먼저 이해해보세요
아래 그림에서 전해 드리는 원리가 아주 중요해요.
이 원리를 이해하신다면 뒤에 나오는 웹 크롤러를 만드는 과정이 한결 쉽게 느껴 질 거예요~~



[그림 설명]

1. 사람이 파이썬 언어로 Selenium 에게 특정 웹 페이지를 크롤링하라고 명령한다.
2. Selenium은 소스코드에 지정된 Web Driver를 실행하여 웹 페이지에 접속한다.
3. 접속한 웹 페이지를 HTML 소스코드 형태로 현재 크롤링을 실행하는 컴퓨터로 가져온다.
4. 수집된 HTML 전체 코드에서 Beautiful Soup를 사용하여 원하는 부분만 골라낸다.
5. 골라낸 데이터를 원하는 형식의 파일로 저장한다.

위 원리가 Selenium을 사용하여 웹 크롤링을 하는 원리입니다.
위 그림에서 중요한 부분은 Selenium 와 Web Driver 와 Beautiful Soup 프로그램이 서로 어떤 역할을 하고 어떤 관계가 있는지를 연계해서 이해해야 합니다.
다소 복잡해 보이지만 실제 실습을 하면서 자세하게 살펴보겠습니다.

2. 웹 크롤링을 위한 설정

이번 챕터에서는 자동 검색 기능을 구현하기 위한 설정을 하고 다음 챕터에서 Beautiful Soup를 설치하고 살펴볼 예정입니다. 아래 순서대로 진행해 주세요.

1) 사람 대신 웹 페이지를 열고 데이터를 수집할 selenium 패키지를 설치합니다.

웹 크롤러를 만드는 다양한 방법이 있는데 이 책에서는 아주 쉽게 작업할 수 있는 selenium (셀레니움) 이라는 패키지를 사용합니다. selenium 패키지는 사람을 대신해서 주어진 작업을 해 주는 조수 프로그램이라고 생각하면 됩니다. **Anaconda를 사용하시는 분은 Anaconda Prompt를 관리자 권한으로 실행하여 아래와 같이 명령을 입력하여 실행하고 파이썬 개별 설치를 하신 분들은 윈도의 명령 프롬프트를 관리자 권한으로 열어서 아래와 같이 명령을 실행하면 자동으로 다운로드 받은 후 설치가 됩니다.**

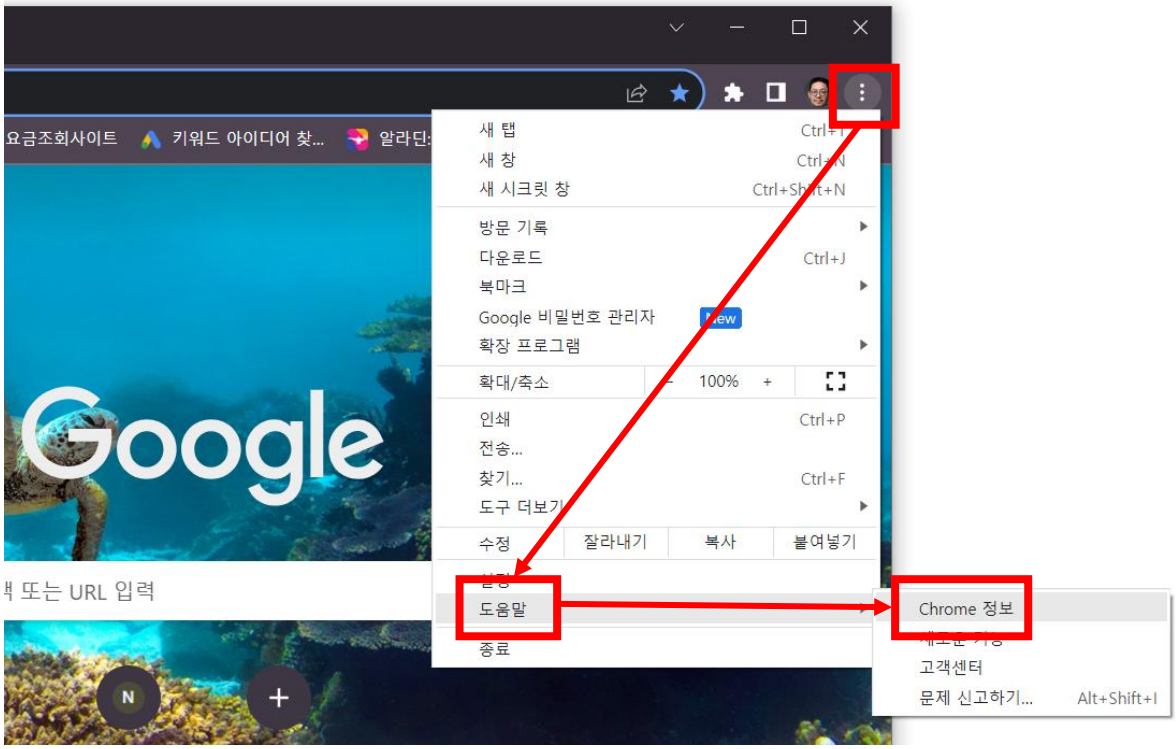
2) selenium 패키지가 사용할 웹 브라우저 프로그램(웹드라이버)을 설치합니다

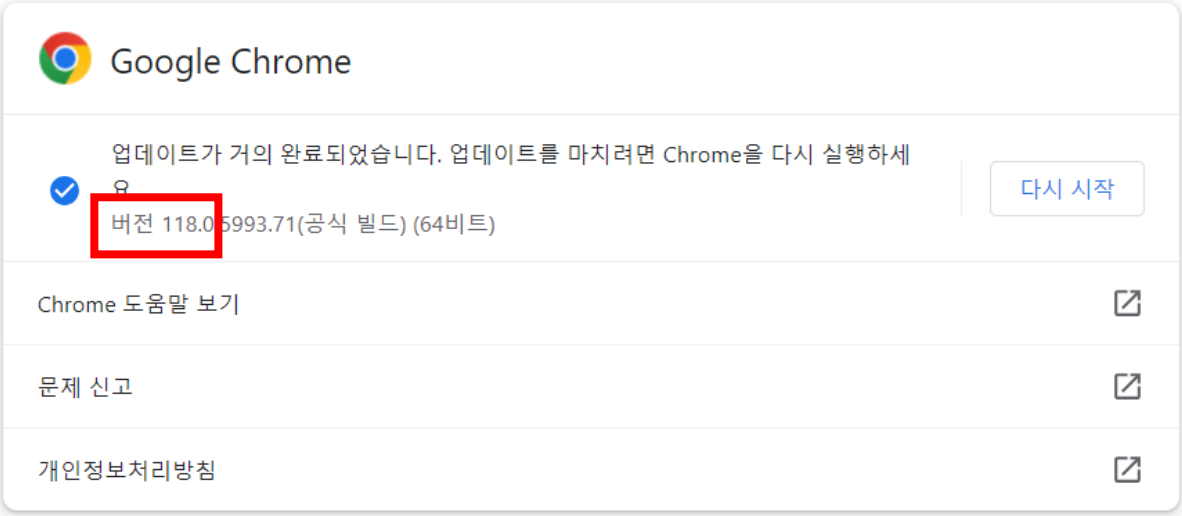
웹 데이터를 크롤링 할 때 사람 대신 selenium이 우리가 지금 설치하는 웹 드라이버 프로그램을 이용해서 웹 페이지를 열고 사람이 지정한 태그를 찾아서 데이터를 수집합니다.
사람이 인터넷을 할 때 웹 브라우저로 인터넷 익스플로러나 크롬, 사파리, Edge 등의 프로그램을 사용하는 것과 동일한 원리입니다. 이 책에서는 웹 드라이버 프로그램으로 google chrome 프로그램을 사용하겠습니다.

꼭 기억해야 할 주의 사항은 chrome 웹 브라우저는 사람이 사용하는 것과 selenium 이 사용하는 프로그램이 다르기 때문에 여러분들의 컴퓨터에 먼저 사람이 사용하는 google chrome 이 설치되어 있어야 합니다. 만약 사람이 사용하는 google chrome 웹 브라우저가 설치되어 있지 않거나 chrome 웹 브라우저와 크롬 웹드라이버가 호환이 되지 않을 경우 웹 크롤링을 진행할 때 "selenium.common.exceptions.WebDriverException: Message: unknown error: cannot find Chrome

binary" 라는 오류가 나오면서 크롬 드라이버가 실행이 안되니까 꼭 기억해 주세요~
이제 selenium 이 사용할 크롬을 아래와 같이 다운로드 해주세요.

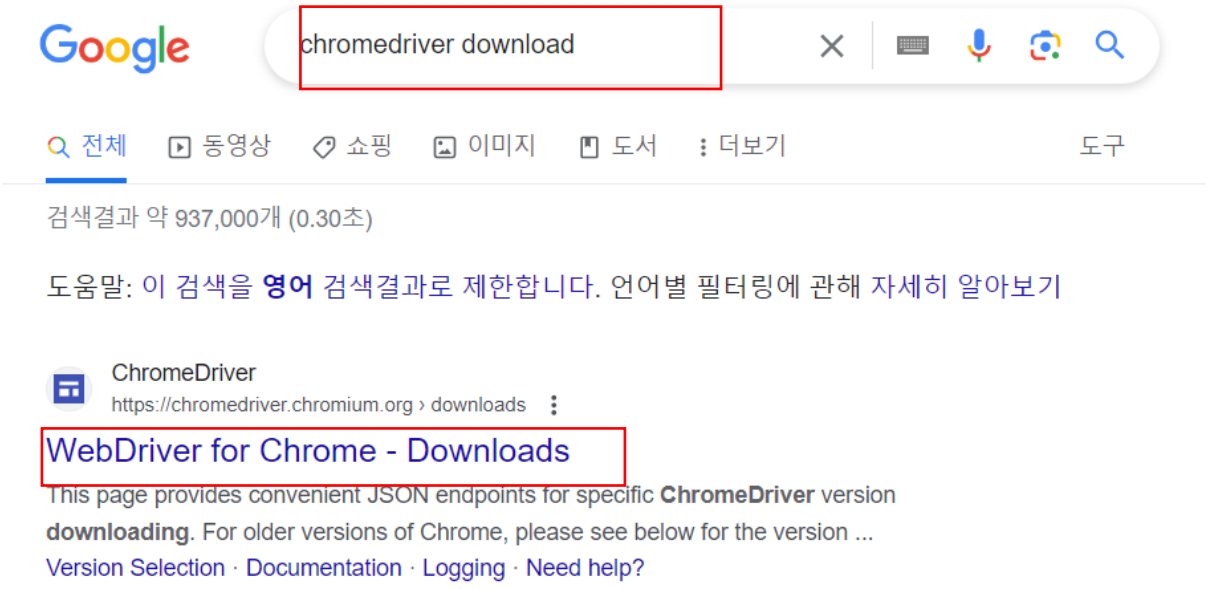
1. 현재 사용하는 컴퓨터에 구글 크롬 브라우저를 설치한 후 아래와 같이 크롬 브라우저의 버전을 확인합니다.



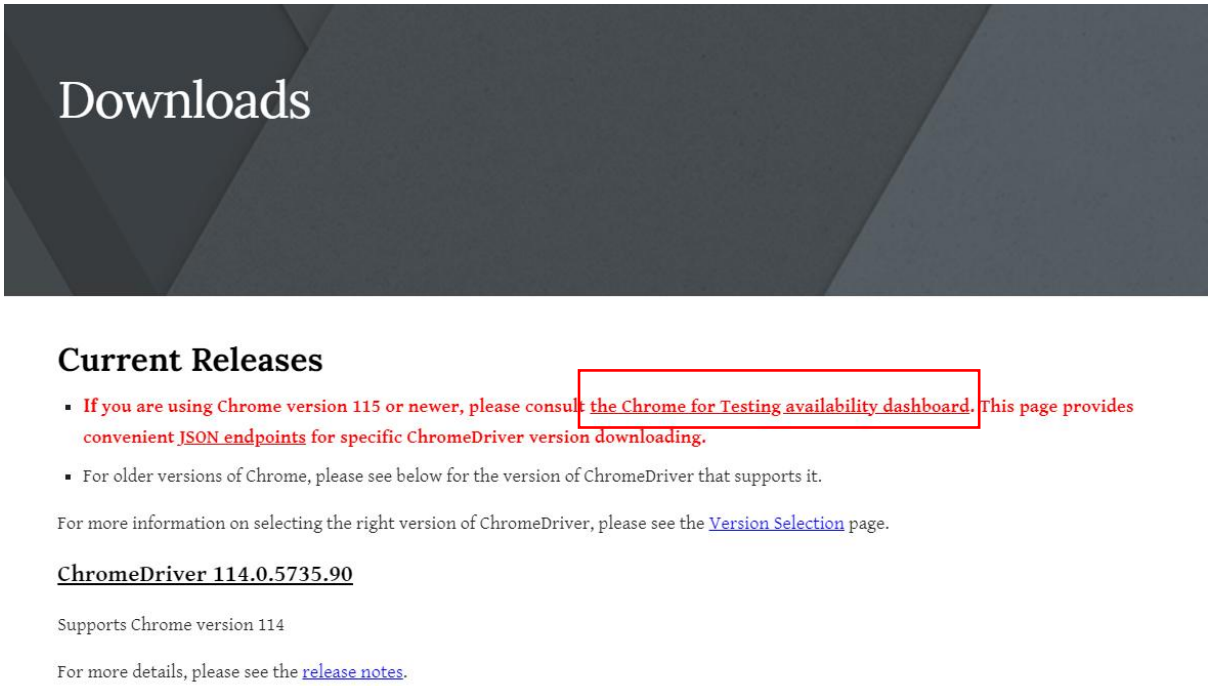


위 그림에서 현재 크롬 브라우저의 버전이 118 임을 확인할 수 있습니다.
(단 이 버전은 현재 학습자 컴퓨터의 버전을 확인 후 사용해야 합니다)
118 뒤에 있는 나머지 숫자는 필요 없으니 무시하셔도 됩니다.

2.아래 그림과 같이 검색 엔진에서 chromedriver download 를 검색합니다.

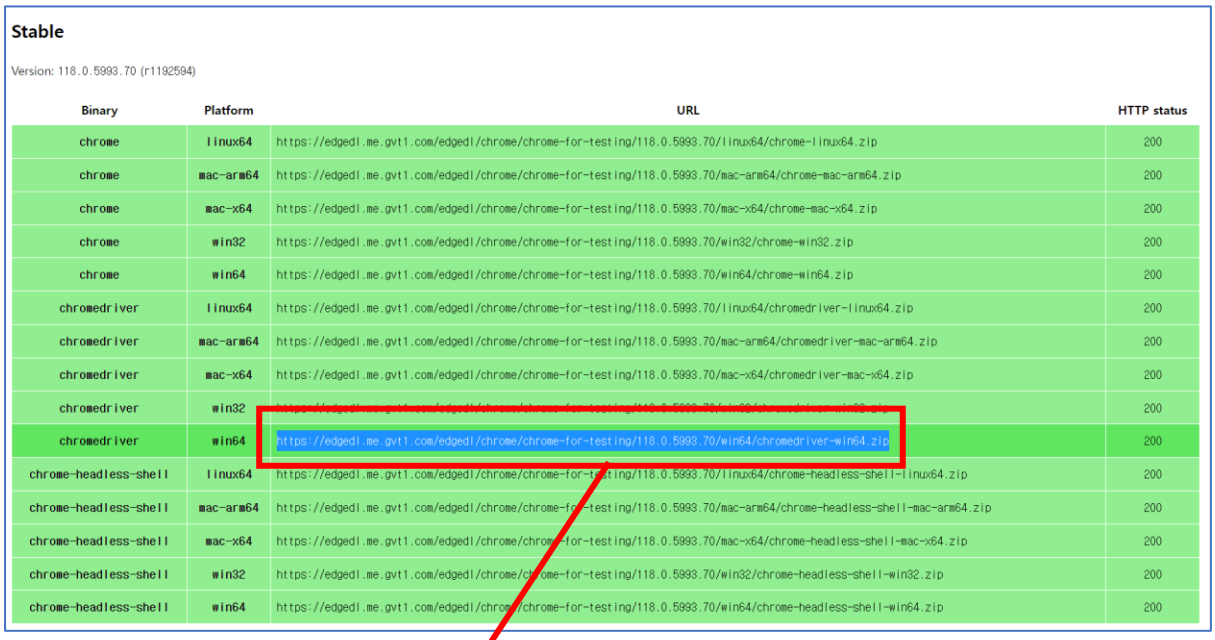


위 그림에서 WebDriver for Chrome – Download 를 클릭하면 아래와 같은 페이지에 접속합니다.



위 화면의 빨간색 부분을 보면 현재 크롬 버전이 115 이상일 경우 빨간색 네모칸 부분의 링크를 클릭해서 다운로드를 받으라고 안내하고 있습니다. 위 빨간색 네모 안의 글씨를 클릭하여 이동합니다.

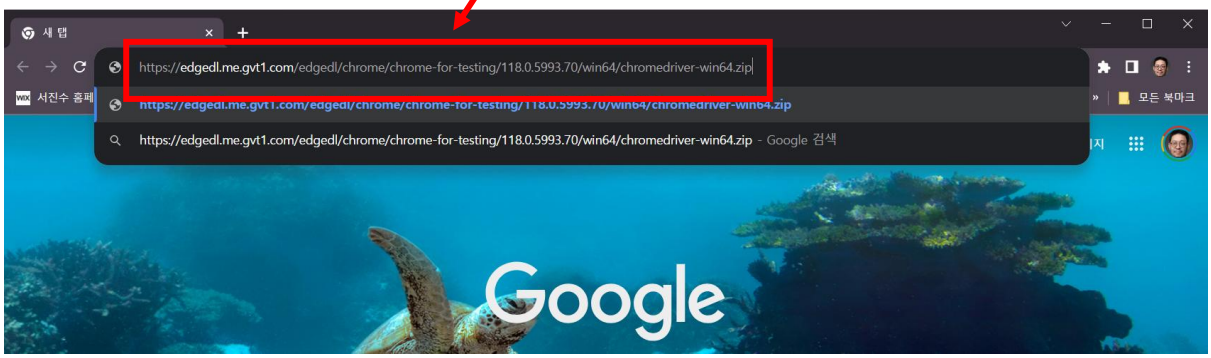
3. 여러가지 항목 중 아래 그림과 같이 Stable 항목 아래에 있는 표에서 해당되는 Platform에 있는 chromedriver URL을 복사하여 웹 브라우저의 주소 입력하는 곳에 붙여넣기 하면 다운로드가 진행됩니다. 저의 경우는 64비트 윈도를 사용하여 win64에 있는 URL을 선택하였습니다.



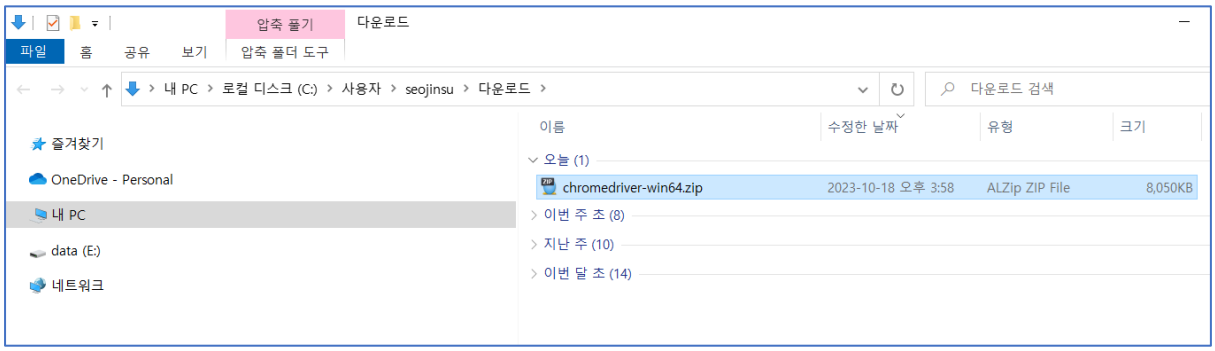
Stable

Version: 118.0.5993.70 (r1192594)

Binary	Platform	URL	HTTP status
chrome	linux64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/linux64/chrome-linux64.zip	200
chrome	mac-arm64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/mac-arm64/chrome-mac-arm64.zip	200
chrome	mac-x64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/mac-x64/chrome-mac-x64.zip	200
chrome	win32	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/win32/chrome-win32.zip	200
chrome	win64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/win64/chrome-win64.zip	200
chromedriver	linux64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/linux64/chromedriver-linux64.zip	200
chromedriver	mac-arm64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/mac-arm64/chromedriver-mac-arm64.zip	200
chromedriver	mac-x64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/mac-x64/chromedriver-mac-x64.zip	200
chromedriver	win32	[illegible]	200
chromedriver	win64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/win64/chromedriver-win64.zip	200
chrome-headless-shell	linux64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/linux64/chrome-headless-shell-linux64.zip	200
chrome-headless-shell	mac-arm64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/mac-arm64/chrome-headless-shell-mac-arm64.zip	200
chrome-headless-shell	mac-x64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/mac-x64/chrome-headless-shell-mac-x64.zip	200
chrome-headless-shell	win32	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/win32/chrome-headless-shell-win32.zip	200
chrome-headless-shell	win64	https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/118.0.5993.70/win64/chrome-headless-shell-win64.zip	200

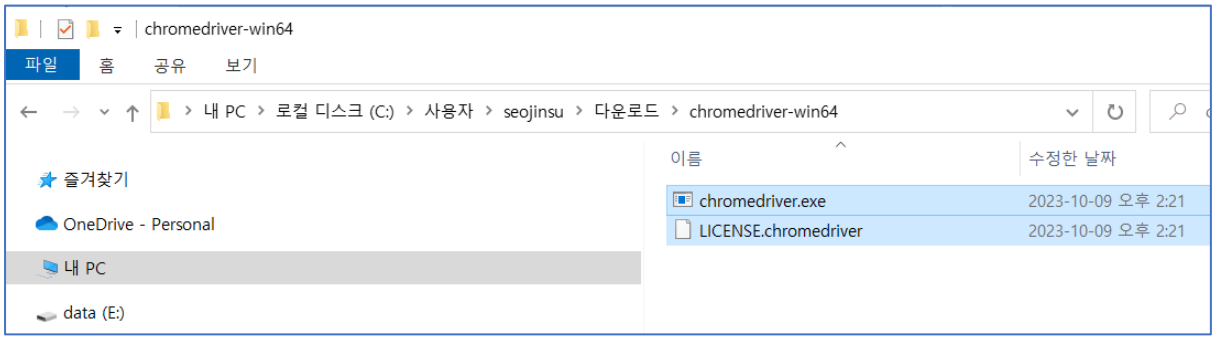


위와 같이 주소창에 URL 링크를 붙여넣기 하면 컴퓨터의 다운로드 폴더에 chromedriver_win64.zip 파일이 다운로드가 됩니다.



4. 다운로드 받은 파일을 압축을 풀어서 c:₩py_temp₩ 폴더 아래로 복사합니다.
(단 반드시 이 경로에 해야 하는 것은 아니지만 우선 실습의 편의성을 위해 위 경로를 사용하겠습니다)

압축을 풀면 아래의 그림과 같이 2개(또는 3개)의 파일이 생성됩니다.



위 그림에 있는 2개(또는 3개)의 파일을 모두 선택하고 복사한 후 c:₩py_temp₩ 폴더 안에 붙여넣기 하면 모든 작업이 완료됩니다.

주의사항:
파일을 모두 복사하여야 하는데 압축 푼 폴더를 복사하는 경우가 간혹 있습니다.
폴더를 복사하면 정상적으로 동작안하고 에러가 발생하니 꼭 파일을 모두 선택 후 복사해주세요.

#참고
크롬 드라이버를 자동으로 업데이트 하는 방법도 있습니다.

```
pip install chromedriver_autoinstaller
import chromedriver_autoinstaller
chromedriver_autoinstaller.install()
driver= webdriver.Chrome()
```

3. 검색창에 검색어를 입력 후 자동 검색하기 기능 구현하기

1) 작업 개요

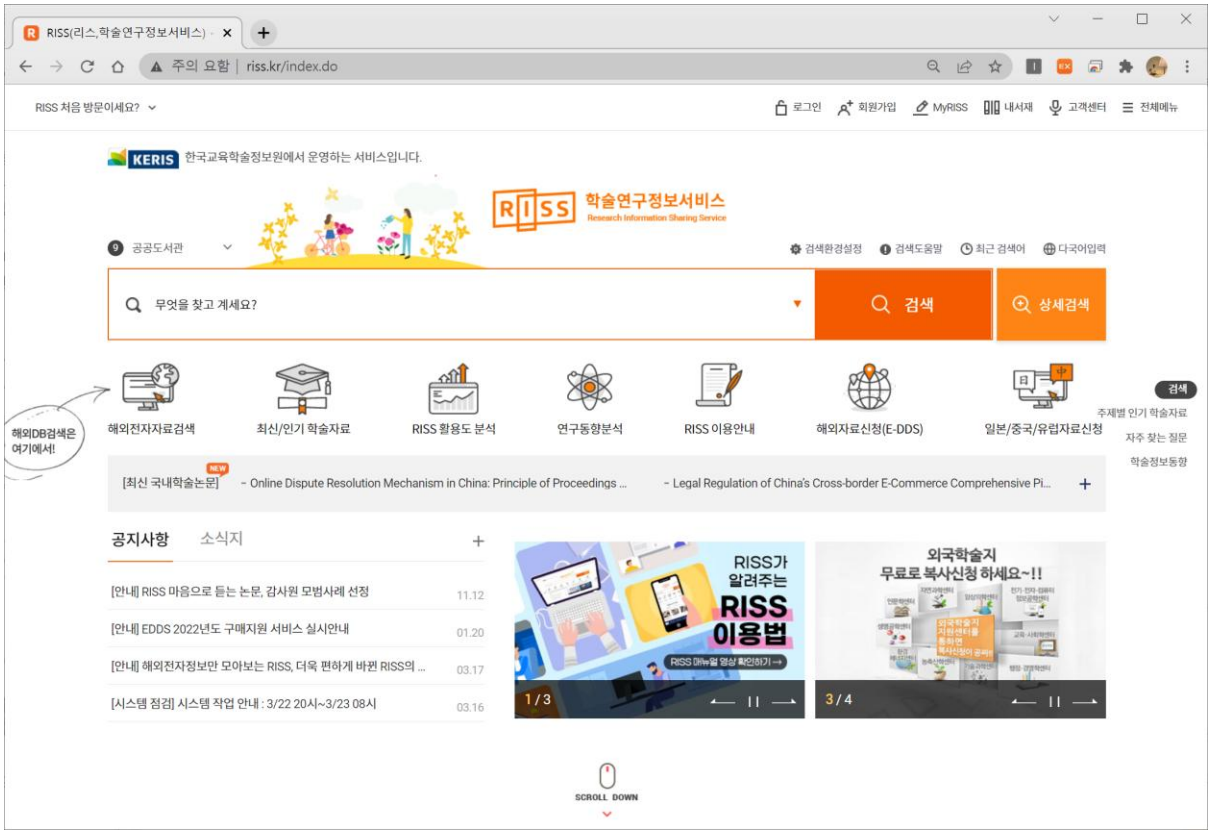
우리가 첫 번째 예제로 배울 내용은 검색창에 검색어를 자동으로 입력한 후 자동으로 검색하여 결과를 출력하게 만드는 것입니다.

즉 selenium 에게 **1.웹 페이지 열어라 -> 2.검색창을 찾아라 -> 3.검색어를 입력해라 -> 4.조회해라** 순서로 작업을 시키는 것이지요.

이 예제에서 반드시 배워야 하는 것은 웹 페이지에서 웹 브라우저의 개발자 도구를 사용해서 특정 메뉴나 항목(엘리먼트)의 이름이나 주소를 찾아내는 방법입니다.

이번 예제에서 사용할 웹 사이트는 한국교육학술정보원에서 운영하는 방대한 논문과 학술자료를 제공하는 대한민국 최고의 학술 정보사이트인 riss 입니다.

웹사이트 주소는 http://www.riss.kr 이고 아래 그림과 같습니다.



참고로 위 사이트의 모습은 수시로 바뀔 수 있습니다.

위 사이트에서 검색창에 특정 키워드를 입력한 후 검색을 실행하면 해당 키워드로 다양한 분야의 방대한 자료들이 조회됩니다.

우리는 selenium을 활용하여 자동으로 위 검색창에 "해양자원" 이라는 키워드를 입력 후 결과를 조회하도록 작업하겠습니다.

2) 일단 실행하기

앞의 작업 개요에서 살펴본 내용으로 작업을 하도록 파이썬으로 코드를 작성해야겠죠?
이제부터 실제 파이썬 코드를 살펴보겠습니다.

```
============================================================
#Chap 1.자동 검색 설정과 자동 검색 수행하기
#Step 1. 필요한 모듈을 로딩합니다

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.chrome.service import Service
import time

#Step 2. 사용자에게 검색 관련 정보들을 입력 받습니다.
print("=" *100)
print(" 이 크롤러는 riss 사이트의 논문 자료 수집용 웹크롤러입니다.")
print("=" *100)
query_txt = input('1.수집할 자료의 키워드는 무엇입니까?(예: 해양자원): ')
print("₩n")

#Step 3. 크롬 드라이버 설정하기
# 방법1. 크롬 드라이버 자동 업데이트 기능 사용하기
#! pip install chromedriver_autoinstaller
import chromedriver_autoinstaller
chromedriver_autoinstaller.install()
driver= webdriver.Chrome()

# 방법2. 크롬 드라이버 경로 수동으로 지정하기
# s = Service("c:/py_temp/chromedriver.exe")
# driver = webdriver.Chrome(service=s)

#Step 4.타겟 사이트에 접속하기
url = 'https://www.riss.kr/'
driver.get(url)
time.sleep(5)
driver.maximize_window()
```

#Step 5. 팝업창이 있을 경우 팝업창 닫기

```
# 팝업 안내창 모두 닫기
main = driver.window_handles
for handle in main:
    if handle != main[0]:
        driver.switch_to.window(handle)
        driver.close()

# 원래 창으로 돌아가기
driver.switch_to.window(driver.window_handles[0])
```

#Step 6. 자동으로 검색어 입력 후 조회하기

```
element = driver.find_element(By.ID,'query')
driver.find_element(By.ID,'query').click( )
element.send_keys(query_txt)
element.send_keys("₩n")
```

==========

위 소스 코드를 Shift + Enter 키를 눌러 실행하면 아래 그림과 같이 크롤링 할 검색어를 입력하는 창이 나오죠?

```
1.수집할 자료의 키워드는 무엇입니까?(예: 해양자원): 해양자원

==========================================================================================
 이 크롤러는 riss 사이트의 논문 자료 수집용 웹크롤러입니다.
==========================================================================================
```

키워드를 입력하는 곳에 "해양자원"을 입력한 후 엔터키를 치면 Chrome 창이 새로 열리면서 위 코드의 18번 행에서 지정한 RISS 웹 페이지가 열리고 검색창에 "해양자원" 을 자동으로 입력한 후 검색하여 결과가 화면에 출력되는 것이 보이죠?

3) 소스 코드 설명

앞에서 실행한 코드를 자세하게 설명 드리겠습니다.

```
1  #Step 1. 필요한 모듈을 로딩 합니다
2  from selenium import webdriver
3  from selenium.webdriver.common.by import By
4  from selenium.webdriver.common.keys import Keys
5  from selenium.webdriver.chrome.service import Service
6  import time
```

위 코드의 4번행에서 selenium 모듈을 import 했습니다.
그리고 아래의 5-7행까지는 웹페이지에서 특정 엘리먼트를 찾거나 글자를 입력할 때 필요한 모듈을 불러오는 부분입니다
. import 명령에 대한 자세한 설명은 이 책의 모듈 부분에 있으니까 참고하세요~

그 후 Step 2의 11번 행에서 사용자에게 검색어를 입력 받아 query_txt 라는 변수에 저장합니다.

```
7  #Step 2. 사용자에게 검색 관련 정보들을 입력 받습니다.
8  print("=" *100)
9  print(" 이 크롤러는 riss 사이트의 논문 자료 수집용 웹크롤러입니다.")
10 print("=" *100)
11 query_txt = input('1.수집할 자료의 키워드는 무엇입니까?(예: 해양자원): ')
12 print("₩n")
```

이렇게 작성 후 Shift+Enter 키를 입력하면 아래와 같이 검색어를 입력 받는 창이 나오는데 "해양자원" 을 입력 한 후 Enter 키를 치세요.

```
1.수집할 자료의 키워드는 무엇입니까?(예: 해양자원): [해양자원]

==================================================================================================
 이 크롤러는 riss 사이트의 논문 자료 수집용 웹크롤러입니다.
==================================================================================================
```

사용자가 입력한 "해양자원" 키워드가 11번 행의 query_txt 변수에 저장되어 있겠죠?
이제 아래 그림처럼 크롬 드라이버를 설정하고 웹 페이지에 접속합니다.

```
13  #Step 3. 크롬 드라이버 설정하기
14  # 방법1. 크롬 드라이버 자동 업데이트 기능 사용하기
15  #! pip install chromedriver_autoinstaller
16  import chromedriver_autoinstaller
17  chromedriver_autoinstaller.install()
18  driver= webdriver.Chrome()
19
20  # 방법2. 크롬 드라이버 경로 수동으로 지정하기
21  # s = Service("c:/py_temp/chromedriver.exe")
22  # driver = webdriver.Chrome(service=s)
```

크롬 드라이버를 설정하는 방법은 자동 설정 방법이 있고 수동 설정 방법이 있습니다.
위 코드에서 15-18 행까지는 자동 설정 방법인데 이 기능을 사용하려면 chromedriver_autoinstaller 라이브러리가 설치되어 있어야 해서 15번 행에 설치하는 명령을 사용했습니다. 그런데 간혹 이 자동 설치 기능이 오류가 생기는 경우도 있어서 수동 설정하는 방법까지 알아 두는 것이 좋습니다.
위 코드의 21, 22번 행이 수동 설정하는 방법입니다.
위 코드의 21번 행의 Service( )에는 chromedriver.exe 파일이 있는 경로를 적어주면 됩니다.
이 책에서는 위 경로이지만 혹시 다른 경로에 chromedriver.exe 파일이 있는 분들은 그 경로로 변경해서 사용하시면 됩니다.

그리고 22번 행에서 webdriver.Chrome( ) 함수로 웹 드라이버는 크롬 드라이버를 사용한다고 지정하면 됩니다. 만약 다른 웹 드라이버를 사용할 경우 이 부분에서 다른 웹 드라이버를 지정하면 됩니다.

```
23  #Step 4.타겟 사이트에 접속하기
24  url = 'https://www.riss.kr/'
25  driver.get(url)
26  time.sleep(5)
27  driver.maximize_window()
```

24번 행에서 우리가 접속할 홈페이지의 주소를 지정한 후 25번 행에 있는 driver.get("URL 주소") 명령으로 주어진 URL 주소의 웹 페이지를 OPEN 합니다. 사람이 웹 페이지를 열기 위해 웹 브라우저를 실행한 후 주소창에 주소를 입력하는 것과 똑같은 효과를 내는 명령어입니다.
여러분들이 크롤링하거나 열기를 원하는 주소로 바꾸고 실행해 보는 것도 재미있겠죠?

26번 행의 time.sleep( )명령은 인터넷 속도에 따라 웹 페이지가 열리는 속도가 다르기 때문에 driver.get( ) 명령으로 웹 페이지를 열도록 시킨 후 다 열릴 때까지 기다리는 명령인데 인터넷 속도가 느릴 경우 이 명령을 사용하지 않으면 에러가 자주 발생할 정도로 아주 중요한 역할을 합니다. 괄호안에 기다릴 시간을 초 단위로 적으면 됩니다.
그리고 27번 행은 chrome driver 로 열린 창을 전체 창 크기로 확대하라는 의미입니다.

```
28  #Step 5. 팝업창이 있을 경우 팝업창 닫기
29  # 팝업 안내창 모두 닫기
30  main = driver.window_handles
31  for handle in main:
32      if handle != main[0]:
33              driver.switch_to.window(handle)
34              driver.close()
35
36  # 원래 창으로 돌아가기
37  driver.switch_to.window(driver.window_handles[0])
```

가끔 접속한 사이트에 팝업 안내창이 여러 개가 열리는 경우가 있는데 팝업창을 모두 닫고 싶을 경우 위 코드를 실행하면 됩니다.

웹 페이지가 열렸다면 이제 키워드를 입력한 후 검색을 해야겠죠?
아래 코드를 볼까요?

```
38  #Step 6. 자동으로 검색어 입력 후 조회하기
39  element = driver.find_element(By.ID,'query')
40  driver.find_element(By.ID,'query').click( )
41  element.send_keys(query_txt)
42  element.send_keys("₩n")
```

위 코드에서 39번 행이 검색어를 입력하는 검색 창 정보를 지정하는 명령입니다.
그리고 40번 행에서 검색 창을 클릭한 후 41번 행에서 검색어를 입력하고 42번 행에서 엔터키를 입력하여 검색을 진행하게 됩니다.

4) 개발자 도구 사용하기

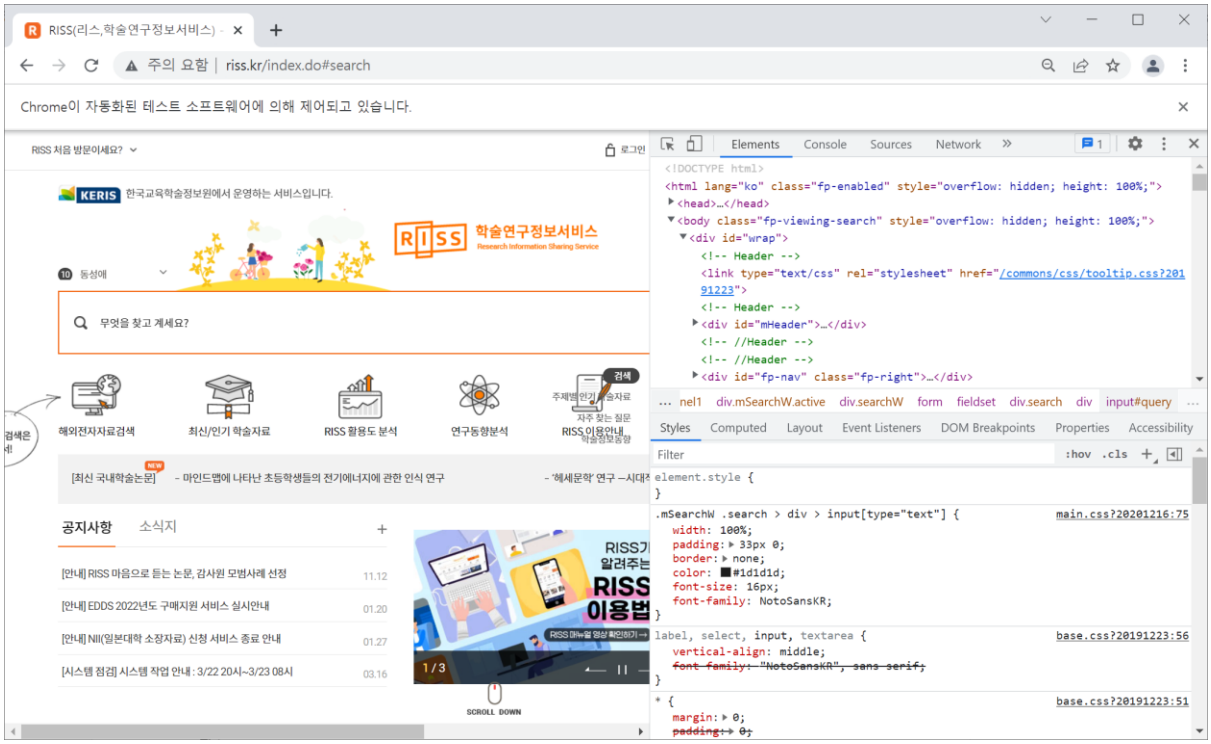
우리는 앞의 실습에서 아래와 같이 입력하고 Shift+Enter를 실행하면 자동으로 크롬이 실행되면서 RISS의 웹 페이지가 열린 후 검색창에 검색어가 입력되는 것을 확인했습니다.

```
38  #Step 6. 자동으로 검색어 입력 후 조회하기
39  element = driver.find_element(By.ID,'query')
40  driver.find_element(By.ID,'query').click( )
41  element.send_keys(query_txt)
42  element.send_keys("₩n")
```

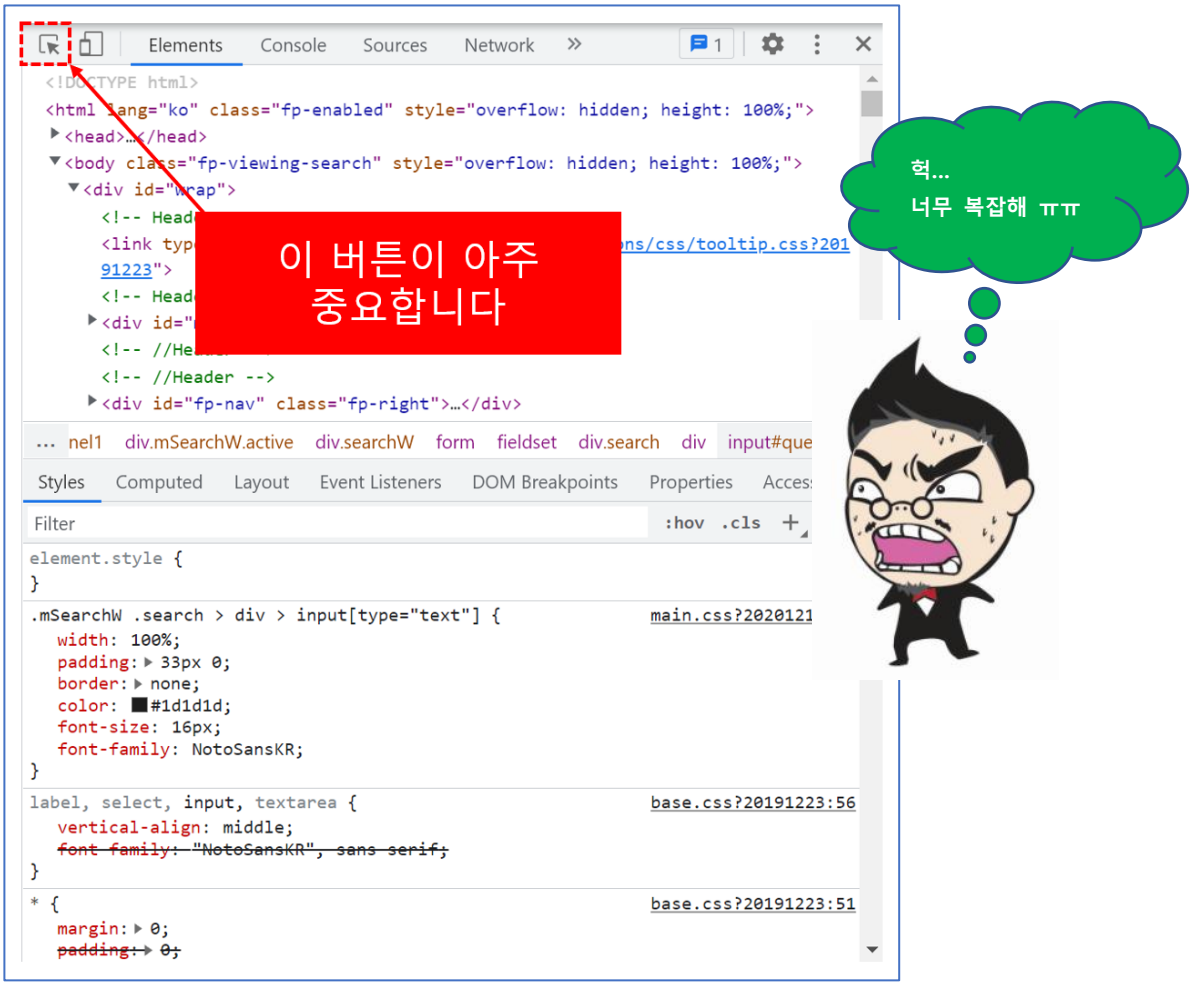
위 코드에서 40번 행에 보면 driver.find_element(By.ID,'query').click( ) 명령이 보이죠?
이 줄의 의미는 selenium 에게 현재 보이는 웹페이지에서 ID값이 "query" 라는 것을 찾아서 클릭하라는 뜻입니다. 여기서 ID값이 "query" 가 과연 무엇이고 어떻게 찾을까요?

앞에서 언급한 작업 순서 중에 현재 페이지에서 검색창에 검색어를 입력한다고 했는데 바로 현재 페이지의 HTML 코드에서 검색창의 ID 값이 query입니다. 이처럼 우리가 웹페이지에서 클릭하거나 값을 입력하는 등의 작업을 하는 대상을 **엘리먼트(element)**라고 합니다.
그럼 검색창의 ID 가 query 라는 것은 어떻게 찾았을까요?
바로 웹 브라우저에서 제공하는 기능인 개발자 도구를 사용해서 쉽게 찾을 수 있습니다.
지금 바로 실습으로 크롬에서 개발자 도구를 이용해서 특정 엘리먼트의 이름이나 ID 등을 찾는 방법을 소개해 드리겠습니다.

먼저 현재 RISS 웹 페이지가 있는 화면에서 F12 번 키나 Ctrl+Shift+I 키를 누르세요.
그러면 아래와 같이 화면이 좌/우로 분할이 되면서 보일 거예요.
(크롬 드라이버일 경우 좌/우로 분할되고 인터넷 익스플로러일 경우 상/하로 분할됩니다)

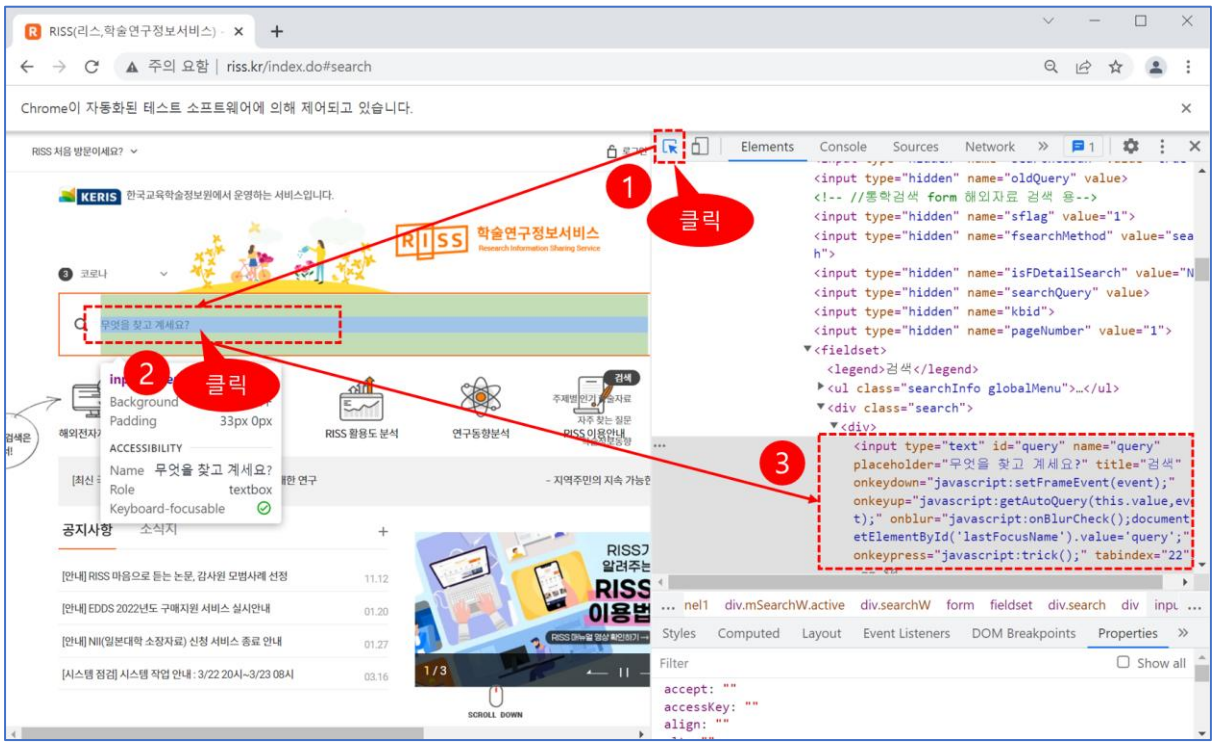


화면이 너무 작아서 잘 안보일 수도 있는데 위 화면에서 오른쪽 화면만 확대해서 다시 보겠습니다.



[파이썬을 활용한 웹 크롤러 만들기- 나하나, 서진수 공저 -]

위 화면이 현재 웹 페이지의 HTML 전체 코드를 보여주는 창입니다.
이 창에서 우리가 원하는 엘리먼트 정보를 많이 찾게 될 거예요.
위 창에서 특정 엘리먼트를 선택하는 버튼을 누른 후 오른쪽 웹 페이지에서 해당 엘리먼트를 선택하면 오른쪽 창에서 해당 엘리먼트의 HTML 코드를 보여줍니다.
말로 하면 어려우니까 아래 그림의 순서를 잘 보세요.



위 그림에서 HTML소스코드가 있는 3번 부분을 보면 id="query" 부분이 보이죠?
이렇게 개발자 도구를 활용해서 특정 버튼이나 메뉴의 name 값이나 id 값을 찾으면 됩니다.
여기서는 해당 엘리먼트의 id 나 name 값이 있어서 둘 중에서 id 값을 사용했는데 만약 id 값이나 name 값이 없을 경우도 있어서 아래의 다양한 방법들을 사용하여 특정 엘리먼트를 클릭하거나 값을 입력할 수도 있습니다.
우선 id 값을 이용하는 방법부터 살펴보고 나머지 방법들은 다른 실습에서 사용하겠습니다.

 # 참고하세요 - 웹 페이지의 특정 element 정보를 지정할 때 많이 사용하는 방법들
- find_element(By.ID, 'id값')
- find_element(By.NAME, 'name값')
- find_element(By.XPATH, 'xpath값')
- find_element(By.LINK_TEXT, 'text값')
- find_element(By.TAG_NAME, 'tag name값')

이번 챕터의 내용들이 앞으로 웹 크롤러를 만들 때 아주 중요하게 사용될 내용들이니까 막히는 부분이 없도록 열심히 연습해 주세요~

4. 연습 문제로 실력 굳히기

1. 네이버 사이트의 검색창에 "**서진수 빅데이터**" 키워드를 입력한 후 검색을 실행하도록 코드를 작성하세요

2. 다음 사이트의 검색창에 "**서진수 빅데이터**" 키워드를 입력한 후 검색을 실행하도록 코드를 작성하세요.

3. 구글 사이트의 검색창에 "**서진수 빅데이터**" 키워드를 입력한 후 검색을 실행하도록 코드를 작성하세요

4. 대한민국 구석구석 사이트(https://korean.visitkorea.or.kr) 에 자동 접속하여 검색창에 "제주도" 키워드를 입력하여 자동 검색하도록 코드를 작성하세요.

5. 가천대학교 홈페이지(https://www.gachon.ac.kr/) 에 자동 접속하여 오른쪽 상단의 돋보기 버튼을 클릭하여 "장학금" 키워드를 입력하여 자동 검색하도록 코드를 작성하세요.

6. 서울대학교 홈페이지(https://www.snu.ac.kr/)에 자동 접속하여 왼쪽 아래부분의 돋보기 버튼을 클릭하여 "장학금" 키워드를 입력한 후 자동 검색하도록 코드를 작성하세요

이번 장에서 배운 내용들을 완전히 습득하셔서 꼭 여러분들의 실력으로 만드세요~

데이터쟁이 서진수가 여러분을 응원합니다~!